

Instructions

Using `checker-obf.jar`

To run the tests, you need to use the checker provided with your project. The project structure given:

```
Project1Starter\  
  lib\  
    checker-obf.jar    <- the checker (do not unzip or move it)  
  src\  
    tests\  
      PublicTests.java <- provided tests that call the checker  
    code\  
      CaveExplorer.java <- your solution
```

The only setup you need is a **one-time step to tell your IDE that** `lib\checker-obf.jar`

is a library on the classpath. Pick your IDE below. You also need **JUnit 5** for the provided tests (see the note at the end).

IntelliJ IDEA

1. Open `MyProject` as a project.
2. If `src` isn't already blue (a source root): right-click the `src` folder → **Mark Directory as** → **Sources Root**.
3. Right-click `lib\checker-obf.jar` in the Project panel → **Add as Library...** → OK.
(Equivalent menu path: **File** → **Project Structure** → **Libraries** → **+** → **Java** → pick the jar.)
4. When you open `PublicTests.java`, IntelliJ shows a prompt to **add JUnit 5 to the classpath** — accept it. Then click the green  next to a test to run it.

Eclipse

1. **File** → **Open Projects from File System...** and select `MyProject`.

2. If `src` isn't a source folder: right-click `src` → **Build Path** → **Use as Source Folder**.
3. Right-click the project → **Build Path** → **Configure Build Path...** → **Libraries** tab →
select **Classpath** → **Add JARs...** → choose `MyProject/lib/checker-obf.jar` → **Apply and Close**.
4. Add JUnit 5: same **Libraries** tab → **Add Library...** → **JUnit** → **JUnit 5** → **Finish**.
5. Right-click `PublicTests.java` → **Run As** → **JUnit Test**.

VS Code

Requires the **Extension Pack for Java** (Microsoft). Install it from the Extensions view if you haven't.

1. Open the `MyProject` folder.
2. In the Explorer sidebar, open the **JAVA PROJECTS** view (bottom-left). Find **Referenced Libraries**, click the **+**, and select `lib\checker-obf.jar`.
 - Equivalently, create `.vscode/settings.json` with:

```
{
  "java.project.sourcePaths": ["src"],
  "java.project.referencedLibraries": ["lib/**/*.jar"]
}
```

The `lib/**/*.jar` pattern picks up the checker (and any JUnit jars you place in `lib`).

3. For JUnit: the **Test Runner for Java** (part of the pack) will let you run the tests with the  gutter icons. If it can't find JUnit, drop the JUnit 5 jars into `lib` (they'll be picked up by the pattern above) or let the extension download them when prompted.

Cursor

Cursor is built on VS Code, so the steps are identical to the **VS Code** section above.

One difference: Cursor installs extensions from the **Open VSX** registry — search there for

"Extension Pack for Java" (or **"Language Support for Java"** + **"Test Runner for Java"**)

and install, then follow the VS Code steps.

Any other IDE / plain command line

The rule everywhere is the same: put `lib\checker-obf.jar` on the **classpath**. From a

terminal in `MyProject` (note the classpath separator differs by OS — `;` on Windows, `:` on macOS/Linux):

Windows (PowerShell):

```
# compile your code + tests against the jar (add your JUnit jars to the -cp too)
javac -cp "lib\checker-obf.jar" -d bin src\code\CaveExplorer.java src\tests\PublicTests.java
```

macOS / Linux:

```
javac -cp "lib/checker-obf.jar" -d bin src/code/CaveExplorer.java src/tests/PublicTests.java
```

(Running the JUnit tests from the command line also needs the JUnit Platform Console

launcher; using an IDE is far simpler for that.)

You also need JUnit 5

`PublicTests.java` uses JUnit 5 (`@Test`, `Assertions`, `assertTrue`). The checker jar does

not include JUnit — add it the same way you'd add any test library (the IDE sections

above each say how). Without it, the test file won't compile.

Verify it's wired up

Open `PublicTests.java`. If `Checker` and `Checker.ValidationResult` resolve with no red underlines and you can run a test, you're done.

Troubleshooting

- **"cannot resolve symbol: Checker" / "ValidationResult"** — the jar isn't on the classpath yet, or the IDE cached an old state. Re-do the add-as-library step, then reload/rebuild the project (IntelliJ: **File** → **Invalidate Caches / Restart**; VS Code/Cursor: **Java: Clean** **Java Language Server Workspace** from the command palette; Eclipse: **Project** → **Clean...**).
- **Do not unzip the jar or copy its contents into `src`**. Leave it in `lib\`; the IDE reads it in place.
- **An `IllegalAccessError` or "module" complaint at runtime** — make sure the project has no `module-info.java`. This project is meant to run on the plain classpath, not the module path.